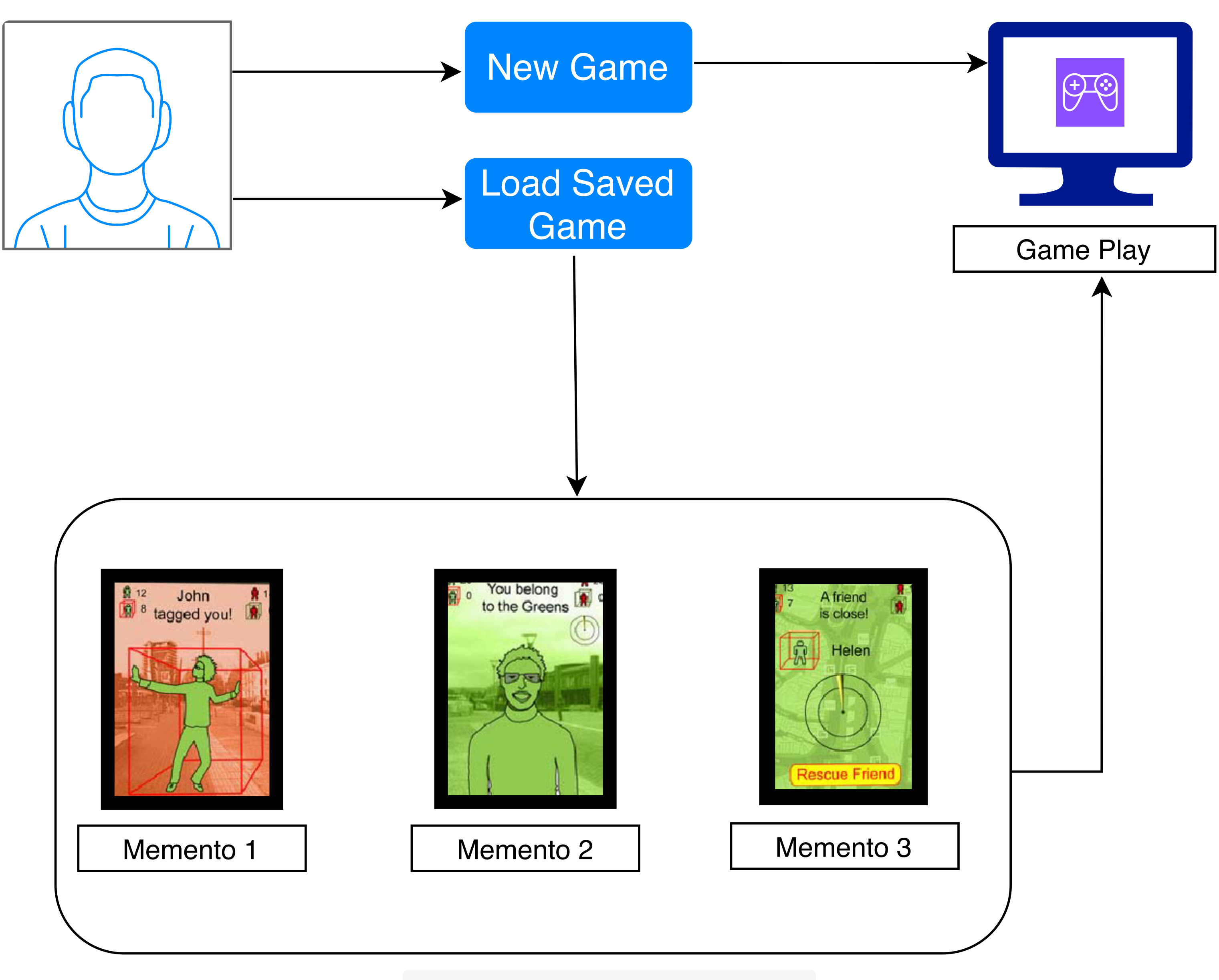
Memento Pattern

The Memento Pattern is a type of behavioral design pattern that can be used to save the state of an object so that it can be restored to that state later without breaking the encapsulation. This pattern is beneficial for implementing undo features.

Real-World Example

Consider a video game in which players progress through multiple levels and gain multiple items, abilities, achievements, and milestones. The problem arises when the player wants to go back to the previous game state, either to explore the game using some different path or to recover from a mistake made in the game. Manipulating the game's internal state directly to enable this could be complicated and risky, potentially corrupting the game data.



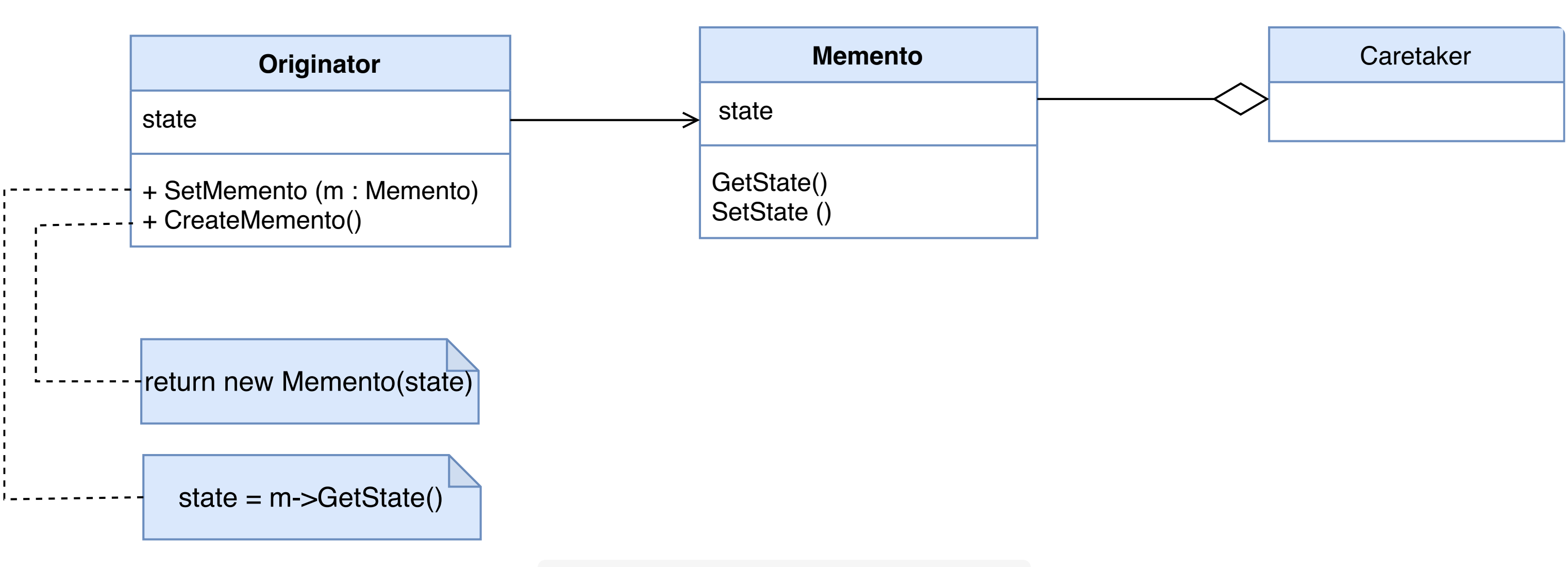
Memento Pattern - Problem Solution

The solution to this problem is hidden in the Memento Pattern. This pattern can save the game state after some time intervals or some milestones. Each memento object represents a game state at a given point in time, as depicted in the image above. Whenever the game player wishes to revert to the previous game state, the game can restore that particular memento object and get that state. Using this approach, we can maintain the integrity of the game's internal structure. Moreover, it simplifies the process of implementing save and load functions.

Structure of Memento Pattern

The class diagram of the Memento Pattern consists of the following key components:

- **Originator**: The primary object whose state has to be preserved and restored. It generates a memento that contains a snapshot of its current internal state. It also uses the memento to return to a previous state.
- **Memento**: This object preserves the originator's state. It's a basic object that usually only contains data and no logic. The originator creates the memento and contains the information needed to restore the originator to its previous state.
- **Caretaker**: The caretaker is in charge of the memento's entire life. It never changes the memento—it just tracks it. The caretaker can request a memento from the originator to save the current state and return a memento to the originator when a rollback is performed.



Memento Pattern - Class Diagram

Implementation of Memento Pattern

In a video game, players advance through different stages while gathering objects and achieving goals. Players must be able to save their progress in the game and return to it whenever they'd like.

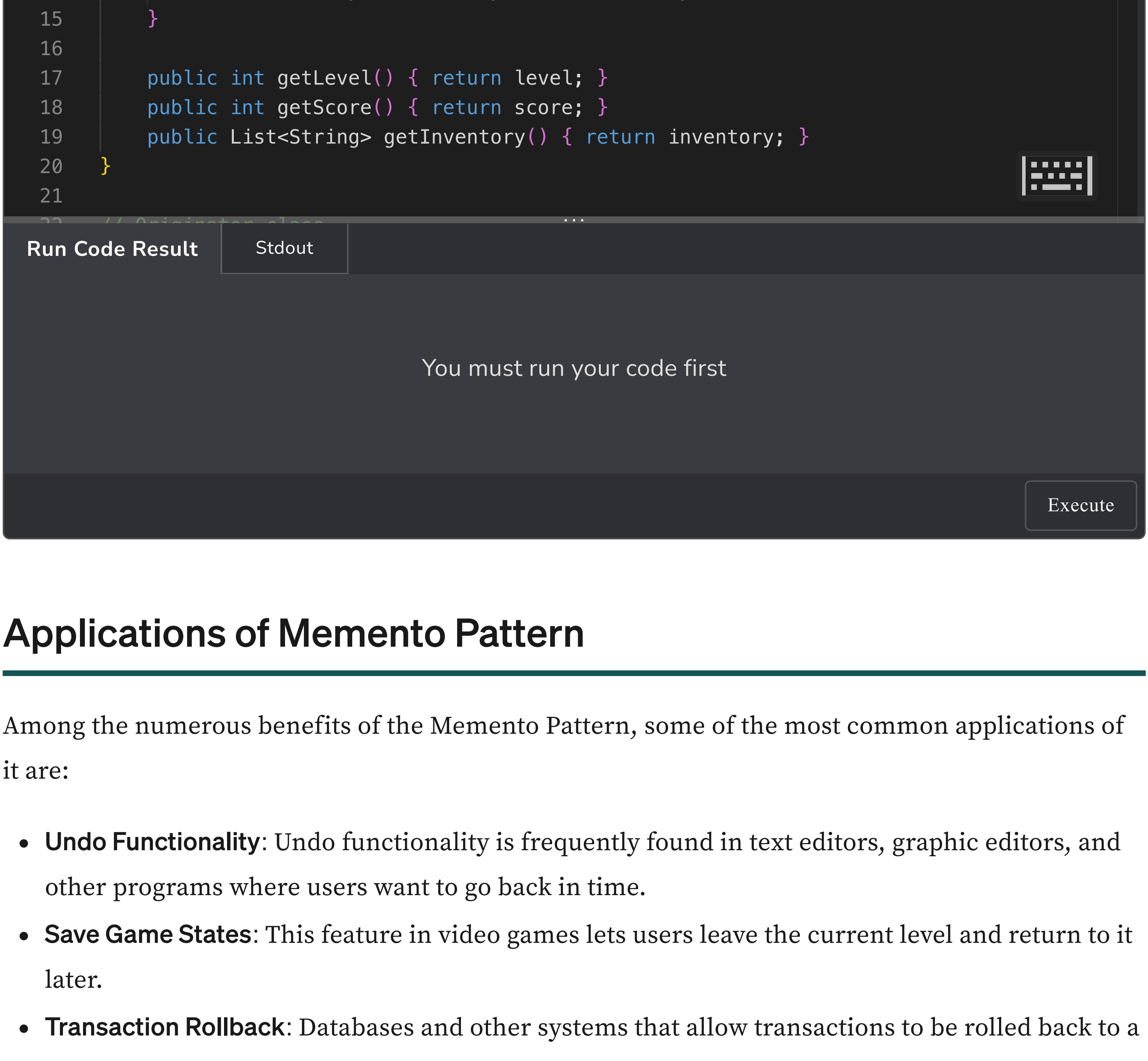
Let's look at the pseudocode for this example:

```
Class Game {
    Private level: Integer
    Private score: Integer
    Private inventory: List of Items
    Private gameSaveManager: GameSaveManager

    Constructor() {
        gameSaveManager = new GameSaveManager()
    }

    Method play(levelIncrement: Integer, scoreIncrement: Integer, newItems: List of Items) {
        level += levelIncrement
        score += scoreIncrement
        inventory.addAll(newItems)
        gameSaveManager.addSave(createSave())
    }

    Method createSave(): GameSave {
        return new GameSave(level, score, inventory)
    }

    Method loadSave(gameSave: GameSave) {
        this.level = gameSave.getLevel()
        this.score = gameSave.getScore()
        this.inventory = gameSave.getInventory()
    }

    Method undoLastPlay() {
        loadSave(gameSaveManager.getLastSave())
    }
}

Class GameSave {
    Private level: Integer
    Private score: Integer
    Private inventory: List of Items

    Constructor(level: Integer, score: Integer, inventory: List of Items) {
        this.level = level
        this.score = score
        this.inventory = inventory
    }

    Method getLevel(): Integer {
        return level
    }

    Method getScore(): Integer {
        return score
    }

    Method getInventory(): List of Items {
        return inventory
    }
}

Class GameSaveManager {
    Private saves: Stack of GameSave

    Method addSave(gameSave: GameSave) {
        saves.push(gameSave)
    }

    Method getLastSave(): GameSave {
        return saves.pop()
    }
}

// Client Code
Main {
    game = new Game()
    game.play(1, 100, [Item1, Item2]) // Play level 1
    game.play(2, 200, [Item3]) // Play level 2
    game.undoLastPlay() // Reverts to the state after playing level 1
}
```

- **Originator**: The primary class that represents the current state of the game. After every play session, it saves its state (level, score, and inventory) and can be restored with a GameSave.
- **GameSave** (**Memento**): Saves a copy of the current state of the game. It contains the inventory, score, and level in this case.
- **GameSaveManager** (**Caretaker**): manages the management of saved games. When a rollback is required, it restores previously saved GameSave objects to the game.

After every play, the Game class's `play` method creates a new save and modifies the current state. The game can be returned to its last saved state using the `undoLastPlay` method. This method allows players to save their progress and return to previous states without exposing the game's complex state management.

Implementation



```java
import java.util.List;
import java.util.ArrayList;
import java.util.Stack;

// Memento class
class GameSave {
    private final int level;
    private final int score;
    private final List<String> inventory;

    public GameSave(int level, int score, List<String> inventory) {
        this.level = level;
        this.score = score;
        this.inventory = new ArrayList<>(inventory);
    }

    public int getLevel() { return level; }
    public int getScore() { return score; }
    public List<String> getInventory() { return inventory; }
}

```

You must run your code first

Applications of Memento Pattern

Among the numerous benefits of the Memento Pattern, some of the most common applications of it are:

- **Undo Functionality**: Undo functionality is frequently found in text editors, graphic editors, and other programs where users want to go back in time.
- **Save Game States**: This feature in video games lets users leave the current level and return to it later.
- **Transaction Rollback**: Databases and other systems that allow transactions to be rolled back to a previous state when necessary are known as transaction rollbacks.
- **Snapshotting**: Making snapshots of system states is useful for various applications, such as virtual machines and complex software systems.

Pros and Cons

Pros	Cons
State Preservation: Safely stores states without exposing the originator's internal structure.	**Memory Usage:** Can consume significant memory if frequent states are saved, especially for complex objects.
Undo/Redo Functionality: Simplifies the implementation of undo/redo mechanisms.	**Complexity:** Adds complexity to the code, particularly in managing the lifecycle of mementos.
High Cohesion: Keeps the state-saving logic within the originator, maintaining high cohesion.	**State Recovery Issues:** Incorrect implementation can lead to inconsistent state recovery.
Decoupling: Caretakers and originators are decoupled, leading to cleaner code.	**Performance Overhead:** Saving and restoring states can be costly in terms of performance, especially for large objects.

The Memento pattern is a powerful design tool for state preservation and undo functionality, allowing for a clear separation of the state-saving logic from the rest of the application. Although it has many advantages, particularly in terms of user experience and system dependability, it also presents complexity and memory management issues. The Memento pattern has advantages, but utilizing them effectively requires careful implementation and state management.

← Previous
Mediator Pattern

Next →
Observer Pattern

☐ Mark as Completed